Image Reconstruction Using Hierarchical Temporal Memory and K-Nearest Neighbor Classifiers

Eyad Elsheita
eyad.elsheita@stud.fra-uas.de
Frankfurt am Main, Germany
1564222

Khaled Kandil
khaled.kandil@stud.fra-uas.de
Frankfurt am Main, Germany
1364635

***Abstract*—Efficient image storage and reconstruction remain critical challenges as digital content volumes grow. This paper presents a cloud-native image reconstruction system that compresses Fashion-MNIST images into Sparse Distributed Representations (SDRs) using a Hierarchical Temporal Memory (HTM) spatial pooler, and reconstructs them using a fusion of K-Nearest Neighbors (KNN) and HTM-based classifiers. Beyond the underlying machine-learning contribution, this report documents the software-engineering process behind the system: a Model Context Protocol (MCP) server that exposes the reconstruction pipeline's individual stages as independently callable tools running in a single Docker container, an Azure Blob Storage layer holding the train/test image sets and persisted classifier models, an Azure Table Storage layer recording per-reconstruction similarity metrics, and a companion AI agent that connects to that server – locally during development or, by changing a single endpoint setting, to an Azure-hosted instance in production – to orchestrate the pipeline through natural-language instructions. The pipeline was trained on 10,000 and evaluated on 2,000 Fashion-MNIST images via an offline batch evaluation of the underlying classifier and fusion logic, not via live requests against the deployed server; the fusion classifier achieved an average vector similarity of 86.22% and binary similarity of 88.12%, outperforming either individual classifier. Equally central to this report is a transparent account of how AI coding assistants were used throughout development, consistent with this course's requirement to disclose and critically review AI-assisted engineering work; the complete, verifiable per-prompt log is maintained separately in the project's PROMPTS.md file, as the guideline permits. We further discuss the limitations of the reconstruction approach, an architectural persistence gap identified during manual review and since closed by persisting classifier models to Blob Storage and reconstruction metrics to Azure Table Storage, the remaining scalability validation this closure still requires, concrete code-quality issues surfaced during review, and lessons learned from combining AI-assisted coding with cloud engineering.**

***Index Terms*—Hierarchical Temporal Memory, Sparse Distributed Representations, Image Reconstruction, K-Nearest Neighbors, Model Context Protocol, Azure Blob Storage, AI-Assisted Software Engineering**

I. Introduction

The volume of digital image data continues to grow at a pace that makes traditional, pixel-level storage increasingly costly to retrieve and manage [1]. A biologically inspired alternative is to encode images into Sparse Distributed Representations (SDRs) using a Hierarchical Temporal Memory (HTM) spatial pooler [2], [3], and to reconstruct the original image from its SDR on demand rather than storing it verbatim. In earlier work by this team, a hybrid reconstruction pipeline combining an HTM-based classifier with a K-Nearest Neighbors (KNN) classifier was shown to reconstruct Fashion-MNIST images with an average similarity above 86% while storing only their sparse column-activation patterns. That earlier work, however, was scoped as a machine-learning evaluation: it did not address how such a pipeline should be engineered, deployed, and operated as a cloud service, nor how AI coding tools were used to build it.

This paper reframes and extends that pipeline as the subject of a software-engineering and cloud-deployment case study. The goals of this work are threefold: (1) package the existing HTM/KNN reconstruction pipeline's individual stages – loading, binarization, spatial pooling, each classifier, denoising, similarity scoring, and PNG conversion – as independently callable tools behind a Model Context Protocol (MCP) server, so that each stage can be driven either directly or by an AI agent, and so that the same agent can reach either a local development instance of that server or an Azure-hosted production instance without any code change (the specific confidence-weighted HTM/KNN fusion recipe reported below remains an offline-only algorithm, not yet one of those live tools); (2) move image storage from the local filesystem to Azure Blob Storage and containerize the service with Docker so it can run on a managed Azure compute target; and (3) document, transparently and verifiably, which parts of the resulting codebase were produced with the help of AI coding assistants, what was changed by hand and why, exactly as required by this course's assessment criteria. This agent-to-Azure-hosted-server relationship is the project's main architectural approach, and it is also the seam along which the system is designed to scale.

The chosen architecture, shown in Fig. 1, favors Azure Blob Storage for the Fashion-MNIST image payloads, using exactly two containers (`train` and `test`) that carry each image through a fixed file-format chain: PNG, binarized text, spatial-pooler (SDR) text, and reconstructed output. The reconstruction pipeline itself is wrapped in Model Context Protocol tools rather than a conventional REST controller, so that the same pipeline stages are addressable both by a human operator and by an LLM-driven agent without duplicating logic. Trained classifiers are held in an in-memory cache for fast repeated access during a running process, backed by

persisted model files in Blob Storage and per-reconstruction metrics in Azure Table Storage, so state is no longer confined to a single process's memory; what this persistence still needs is validation under an actual multi-replica deployment. Docker is used to keep the deployment target-agnostic, matching the containerized deployment pattern shown in the course's reference example.

The remainder of this paper first reviews the related work underlying the reconstruction algorithm and positions this paper's contribution relative to it, then describes the system design, cloud architecture, and AI-assisted development workflow, then reports reconstruction accuracy, computational performance, and deployment evidence, then documents AI tool usage in the structured format required by this course, then discusses limitations, threats to validity, and lessons learned, and finally concludes with a summary and future work.

II. Related Work

Large-scale data growth from transactional, sensor, and behavioral sources motivates compact, efficient storage strategies [1]. The Spatial Pooler algorithm addresses this by modeling how the neocortex converts input patterns into Sparse Distributed Representations: input patterns that activate overlapping sets of neurons are grouped into a common output representation [2], [3]. HTM builds on this by learning temporal sequences of SDRs using an online, unsupervised Hebbian-style learning rule, organizing neurons into columns with a dual sparse coding scheme at the column and cell level, which gives the model robustness to noise and the ability to represent multiple simultaneous predictions [4]. KNN, by contrast, is one of the simplest classifiers in the literature, classifying a test sample by the majority vote of its nearest neighbors under a chosen distance or overlap measure, and remains competitive with far more complex classifiers on many tasks [5].

These works establish the algorithmic building blocks used in this paper's reconstruction pipeline, but none of them address the engineering concerns this course requires: packaging an ML pipeline as a cloud-hosted, tool-callable service, or disclosing how AI coding assistants contributed to that engineering work. The Model Context Protocol [8] is used here specifically to standardize how an external agent can discover and invoke the pipeline's stages as tools, and Azure Blob Storage [9] provides the managed persistence layer for image payloads; both are treated in this paper as engineering building blocks rather than research contributions in themselves.

Positioning relative to the earlier, ML-focused version of this work: that evaluation established that a fusion of HTM and KNN classifiers reconstructs Fashion-MNIST images more accurately and more consistently than either classifier alone, but it was executed and measured entirely on local hardware, with no externally callable interface and no discussion of how the implementation itself was produced. The present paper treats those results as an established baseline and focuses its contribution on the surrounding engineering: how the same pipeline is packaged, deployed, operated, and — per this course's explicit requirement — how its construction with AI coding assistance is disclosed and reviewed.

The `ImageReconstructionAgent`'s design — an LLM that decides which MCP tool to call based on a natural-language request — follows the broader pattern of tool-augmented language models established by ReAct [11], which interleaves explicit reasoning traces with discrete actions so a model can decide *when* to act rather than only *what* to say, and Toolformer [12], which showed that a language model can learn which external tool to invoke and how to format the call without exhaustive supervision. This paper does not contribute to that line of research; it applies an existing, off-the-shelf agentic pattern (via `Microsoft.Agents.AI` and the Model Context Protocol) to a concrete reconstruction pipeline, and focuses instead on the engineering and disclosure obligations that come with operating such an agent against a cloud-hosted backend.

III. System Design and Methodology

A. Tech Stack and Architecture Overview

The service is implemented in C# on .NET 9. The core component, `Image_Reconstruction_Classifier`, is an ASP.NET Core application hosted as a Model Context Protocol server (via the `ModelContextProtocol.AspNetCore` SDK) that runs as a single process inside one Docker container and exposes eight tools, one per pipeline stage. A companion console application, `ImageReconstructionAgent`, uses `Microsoft.Agents.AI` together with an OpenAI chat model (`gpt-4o-mini` by default) to connect to the MCP server over HTTP and drive the pipeline through natural-language instructions — training the classifiers, reconstructing images, and evaluating similarity. The server endpoint the agent connects to is a single configuration value, `MCP_SERVER_URL`: it defaults to a local instance (`http://localhost:5280`) for development, and is repointed at an Azure-hosted instance for production use, with no change to the agent's code. This agent-accesses-tools-via-Azure relationship is the main approach of the delivered product – the agent never touches the reconstruction pipeline directly, only ever through this MCP boundary. The agent is itself an architectural component of the delivered product, distinct from the AI coding assistants used to help write the code, which are addressed separately below.

Three non-functional goals shaped these choices. *Portability*: Docker keeps the service host-agnostic, so the same image runs unmodified on a developer's machine or on Azure. *Extensibility*: each pipeline stage is a separately versionable MCP tool (Table I) rather than a monolithic handler, so a stage can be changed or replaced without touching the others. *Operability*: structured logging via `CloudLogger.cs` (timestamped, leveled log lines tagged by tool name) and per-run Excel exports via `ExcelHelper.cs` give a human-readable audit trail without requiring a full observability stack, though neither substitutes for the metrics/tracing pipeline a production deployment would eventually need.

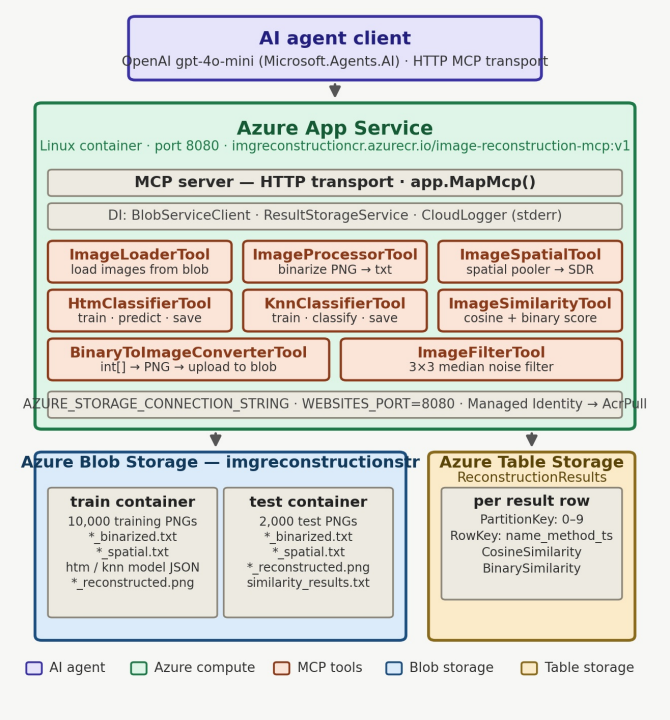

Fig. 1. Deployed architecture: an AI agent client calls MCP tools on the containerized MCP server (Azure App Service), which reads/writes Fashion-MNIST images and persisted classifier models to Azure Blob Storage's `train` and `test` containers, and writes per-reconstruction similarity metrics to the `ReconstructionResults` table in Azure Table Storage via `ResultStorageService`.

As Fig. 1 shows, the MCP server exposes its eight tools behind a single in-process boundary and keeps trained HTM classifiers in an in-memory dictionary (`Dictionary<objectType, MyHtmClassifier>`) for fast repeated access during a running process; trained HTM/KNN model state is additionally persisted as JSON files in the Blob `train` container via each classifier tool's own save/load methods, so a classifier's trained state is no longer confined to that single process's memory. Table I lists the eight MCP tools and the pipeline responsibility each one wraps, and Fig. 2 traces the exact sequence of tool calls a single reconstruction request makes through this boundary.

A single reconstruction request can flow through the server as follows: (1) the caller selects a test image and invokes `ImageLoaderTool`/`ImageProcessorTool` to obtain its binarized vector; (2) `ImageSpatialTool` converts that vector into an SDR; (3) `HtmClassifierTool` and `KnnClassifierTool` each produce a candidate reconstruction from the SDR, drawing on the in-memory classifier cache; (4) either candidate can be denoised via `ImageFilterTool`'s median filter and rendered as a PNG via `BinaryToImageConverterTool`; and (5) `ImageSimilarityTool` scores a reconstruction against the original image. Each of these steps is a distinct, independently testable MCP tool call, which is what lets a human, a test harness, or the `ImageReconstructionAgent` exercise the HTM and KNN classifiers separately today. Combining the two classifiers' outputs into the single fused reconstruction reported in this paper's results – confidence-weighted averaging plus Gaussian-weighted local voting – is *not* currently one of those live tool calls or an agent-orchestrated sequence; that specific fusion algorithm exists only in the offline `LocalPipelineRunner.cs` batch

TABLE I
MCP TOOLS EXPOSED BY THE IMAGE_RECONSTRUCTION_CLASSIFIER SERVER

MCP Tool	Responsibility
ImageLoaderTool	Reads binarized image files and flattens them into 1-D input vectors.
ImageProcessorTool	Binarizes raw grayscale images at a fixed pixel threshold.
ImageSpatialTool	Runs the NeoCortexApi spatial pooler to produce each image's SDR.
HtmClassifierTool	Trains/queries the HTM classifier and produces weighted-average pixel reconstructions.
KnnClassifierTool	Trains/queries the KNN classifier over SDR overlap, and persists/reloads its model to and from Blob Storage.
ImageSimilarityTool	Computes cosine and binary similarity between original and reconstructed images.
ImageFilterTool	Applies a 3×3 median filter to a single binary image to reduce noise (does not itself perform HTM/KNN fusion; see below).
BinaryToImage-ConverterTool	Converts the fused binary reconstruction back into a viewable PNG image.

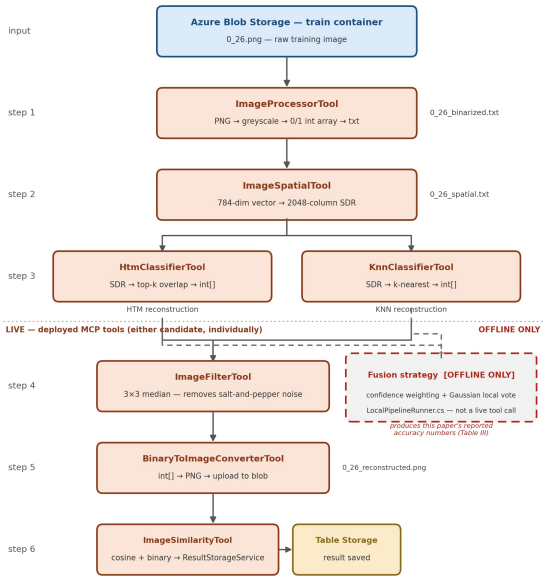

Fig. 2. Per-request tool flow: a single image travels from the Blob `train` container through binarization, spatial pooling, and dual HTM/KNN classification – all live MCP tool calls, shaded orange. Either candidate can then be denoised, converted, and scored (also live). The dashed, separately shaded fusion step is *not* a live tool call; it exists only in the offline `LocalPipelineRunner.cs` batch pipeline and is shown here to make that distinction explicit rather than implying it happens in the deployed request path.

pipeline described further below.

1) MCP Tool Interface Design: Each tool method is annotated with `[McpServerTool]` plus a natural-language `Description` attribute on the method and on every parameter (e.g., `HtmClassifierTool.TrainClassifier` is described as training "the HTM classifier for a specific object type (0-9) using spatial SDR blobs…paired with their matching binarized original images…"). These descriptions are not documentation for a human reader; they are the interface the LLM agent actually uses at run time to decide which tool to call and how to fill its parameters from a natural-language request, which is what makes the ReAct/Toolformer-style tool-calling pattern discussed in Section II concretely work here. Several tools expose more than one MCP-callable method rather than a single entry point — `ImageLoaderTool` offers both a bulk `LoadImagesFromBlob(containerName, numberOfImages)` and a single-image `LoadSingleImageFromBlob`, `HtmClassifierTool` offers both a per-class `TrainClassifier(containerName, objectType, maxImages)` and an all-classes variant, and `KnnClassifierTool` exposes four methods covering training (`TrainKnnClassifier`), prediction (`PredictWithKnn`), and explicit Blob-backed model persistence (`SaveModelToBlob`/`LoadModelFromBlob`) — giving the agent a choice of granularity depending on the request. Tool parameters are restricted to primitive and array types (`string`, `int`, `int[]`) rather than custom objects, which keeps the schema the LLM must populate simple and reduces the chance of malformed tool calls.

A second, previously unstated architectural consequence follows directly from the tool signatures: every stage-specific tool (`ImageProcessorTool`, `ImageSpatialTool`, `HtmClassifierTool`, `KnnClassifierTool`, `BinaryToImageConverterTool`) both downloads its input from and uploads its output back to the *same* `containerName` in Azure Blob Storage, rather than passing intermediate arrays directly between tool calls in memory. Blob Storage is therefore not only the pipeline's image store, as Fig. 1 shows, but also its de facto inter-tool data-exchange medium: each stage's output must round-trip through Blob Storage before the next tool call can consume it. This keeps individual tool calls stateless and independently retriable, which is valuable for an LLM-driven caller that may retry a failed step, but it also means every pipeline stage pays a network round-trip to Blob Storage that a purely in-process pipeline would not, a cost not captured in this paper's local baseline measurement.

B. Cloud Architecture and Deployment

Source and test images are held in Azure Blob Storage (storage account `imgreconstructionstr`, resource group `RG-ImageReconstruction`, region Germany West Central) under exactly two data containers, `train` and `test` (Fig. 1), alongside the platform-managed `$logs` container Azure creates automatically; the `train` container additionally holds the persisted HTM/KNN model

JSON described above. Azure Table Storage is used for reconstruction metrics: `ResultStorageService` writes each result to a `ReconstructionResults` table via `Azure.Data.Tables` [10], keyed by `PartitionKey` (object type, 0–9) and a composite `RowKey` of name, method, and timestamp, with `CosineSimilarity` and `BinarySimilarity` stored per row. Per-image similarity metrics are also exported via `ExcelHelper.cs` for offline analysis, so the two persistence paths coexist rather than one replacing the other.

The MCP server is packaged with a multi-stage Dockerfile that builds on `mcr.microsoft.com/dotnet/sdk:9.0` and runs on the smaller `mcr.microsoft.com/dotnet/runtime:9.0` image. It is deployed to Azure App Service for Containers (Linux) as the Web App `image-reconstruction-application` (App Service plan `AnnApiPlan`, Basic B1), pulling the image `imgreconstructioncr.azurecr.io/image-reconstruction-mcp:v1` from Azure Container Registry `imgreconstructioncr` via a system-assigned Managed Identity granted the `AcrPull` role rather than embedded registry credentials. The app listens on the port declared by the `WEBSITES_PORT=8080` app setting, and the Blob Storage connection string is supplied via the `AZURE_STORAGE_CONNECTION_STRING` app setting rather than being hardcoded, so no connection secret lives in source control. No automated health check, monitoring, or CI/CD deployment pipeline exists yet for the containerized service.

C. Image Reconstruction Pipeline

Raw Fashion-MNIST images (28×28 grayscale) are binarized at a threshold of 128 and flattened into a 1-D array (`ImageProcessing.cs`, `ImageLoader.cs`). The spatial pooler (`ImageSpatial.cs`, via NeoCortexApi [7]) is configured with an input dimension of 784 and a column dimension of 2048, a potential-connection percentage of 0.8, an inactive synaptic-permanence decrement of 0.008, and an active increment of 0.05, producing a set of active column indices that form each image's SDR. Fig. 3 summarizes the training-side data flow from raw dataset to trained classifiers.

Two classifiers are trained on these SDRs per garment class. The KNN classifier (`KnnClassifier.cs`) stores SDR/label pairs; at prediction time it computes the SDR overlap between the test sample and each stored neighbor, weights each neighbor's vote by the square of that overlap (so higher-similarity neighbors dominate disproportionately), and assigns the label with the highest aggregated weighted score:

$$w_i = o_i^2, \qquad L = \arg\max_{\ell} \sum_{i\,:\,y_i=\ell} w_i \tag{1}$$

where o_i is the SDR overlap between the test sample and training neighbor i, and y_i its label.

The HTM classifier (`HtmClassifier.cs`) instead reconstructs each pixel directly: for a given pixel j, its predicted value $\hat{p}_j$ is the weighted average of that same pixel across the

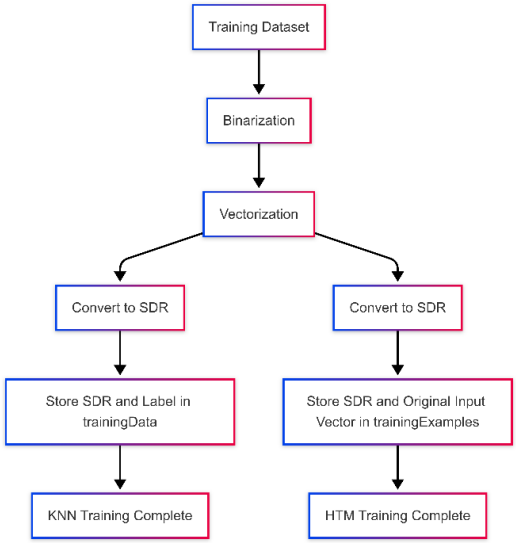

Fig. 3. Classifier training pipeline: binarization, vectorization, and SDR generation for both classifiers.

k most similar training images, again weighted by squared SDR overlap and normalized by the sum of those weights:

$$\hat{p}_j = \frac{\sum_{i=1}^{k} o_i^2\, p_{j,i}}{\sum_{i=1}^{k} o_i^2} \tag{2}$$

with a 0.5 threshold applied to $\hat{p}_j$ for the final binary output.

At inference time, both classifiers are queried and their binary and vector-valued outputs are fused (Fig. 4): vector outputs are combined using cosine-similarity weighting, while binary outputs go through Gaussian-weighted local voting to resolve pixel-level disagreements, followed by median filtering to suppress residual noise. This entire fusion step, exactly as evaluated later in this paper, is implemented in `LocalPipelineRunner.cs` and runs as an offline batch job over local files – it is *not* a live MCP tool call. The individual classifiers (`HtmClassifierTool`, `KnnClassifierTool`) and a generic median filter (`ImageFilterTool`) are separately exposed as live tools (Table I), but no live tool or agent-orchestrated sequence currently reproduces the confidence-weighted combination step itself; closing that gap is listed as future work at the end of this paper. The fused binary result is converted back into a PNG via `BinaryToImageConverter.cs` within that offline pipeline, and both cosine and binary similarity scores are computed against the ground-truth test image and exported to Excel via `ExcelHelper.cs` for offline analysis.

D. AI-Assisted Development Workflow

Two distinct forms of AI are present in this project and are treated separately, as this course requires. The first is the `ImageReconstructionAgent` described in Section III-A: an AI agent that is a feature of the delivered product, used at run time to operate the pipeline. The second is the use of AI coding assistants (e.g., GitHub Copilot, ChatGPT, or

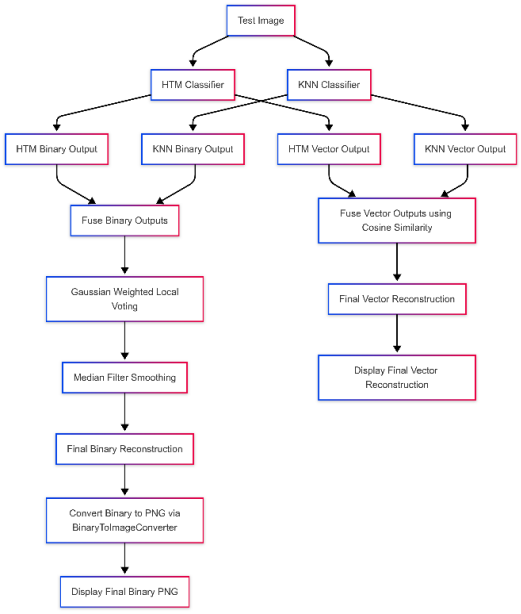

Fig. 4. Test-time reconstruction and fusion pipeline combining HTM and KNN outputs.

Claude) during development to help write, scaffold, or debug the code itself — which this course requires to be disclosed per-contribution, independent of whether the resulting code touches AI at run time.

At a concrete level, AI coding assistance shaped several specific engineering decisions in this codebase, each disclosed with its own prompt, output, and manual review step in `PROMPTS.md`: proposing the overall cloud-migration architecture (Blob Storage in place of the local filesystem, an MCP tool layer over the pre-existing classifier classes, App Service, ACR, and Table Storage); establishing the `[McpServerToolType]`/`[McpServerTool]` pattern behind every tool in Table I; designing the Blob Storage integration (temporary-folder download/process/upload cycle, prefix-filtered blob listing) and the resulting two-container layout; diagnosing and fixing an Azure SDK signature mismatch in `GetBlobsAsync` across four tool files; designing the model-persistence mechanism (`ExportTrainingExamples`/`ImportTrainingExamples`, JSON serialization of SDR sets); designing `CloudLogger` and specifically identifying that log output had to go to `stderr` rather than `stdout` so it would not corrupt the MCP protocol stream; designing the `ReconstructionResults` Table Storage schema from Section III-B and catching an OData filter-injection risk in an early draft of its query filter; and guiding the multi-stage Dockerfile and Azure App Service deployment sequence, including the switch from stdio to HTTP transport that a platform-hosted MCP server requires. None of these were accepted as single-shot output: each required the team to verify the proposal against the actual SDK/protocol behavior, adapt it to this codebase, and in at least the injection-risk and transport cases, correct a real defect in the first draft.

TABLE II
CODEBASE FOOTPRINT BY PROJECT (MEASURED VIA `wc -l`, EXCLUDING BUILD ARTIFACTS)

Project	Files	LOC	Largest File
`Image_Reconstruction_Classifier`	23	3,118	`LocalPipelineRunner.cs` (526)
`ImageReconstructionAgent`	1	90	`Program.cs` (90)

A full per-component breakdown of AI-generated versus hand-written code beyond these highlights requires the team's actual commit history, which belongs in `PROMPTS.md` rather than being estimated here. One smaller artifact is worth flagging as a concrete starting point for that review, independent of who or what produced it: `Properties/launchSettings.json` defines a `Training_Image_Spartial` environment variable – a misspelling of *Spatial* – alongside absolute, machine-specific Windows paths that will not resolve on any other machine or inside the Docker container described in Section III-B. This should be corrected and the paths externalized (e.g., to `appsettings.json` or environment variables injected at deploy time) regardless of its origin; whether it traces back to an AI suggestion or manual entry is exactly the kind of detail the per-prompt log in `PROMPTS.md` should settle. A previously-noted second issue – two tool files breaking the project's otherwise consistent PascalCase file-naming convention – has since been renamed (`Tools/ImageProcessorTool.cs` and `Tools/BinaryToImageConverterTool.cs` now follow the convention) and is not carried forward here as still-open.

The structured, per-prompt disclosure required by the course (prompt text, tool and version, raw AI output, manual changes, and iteration count) is provided in the project's `PROMPTS.md` log, referenced there by commit hash.

E. Codebase Footprint and Dependencies

For transparency, this subsection reports the codebase's actual size and third-party dependencies as measured directly from the repository rather than estimated. The `Image_Reconstruction_Classifier` MCP server comprises 23 C# source files and 3,118 lines of code (excluding generated `obj`/`bin` artifacts), while the companion `ImageReconstructionAgent` console application is a single 90-line `Program.cs`, reflecting its role as a thin orchestration layer that delegates all reconstruction logic to the MCP server rather than duplicating it. Table II breaks this down further.

Two details in Table II are worth calling out. First, the single largest file in the server project is `LocalPipelineRunner.cs` (526 lines), a standalone driver that runs the full train/test/evaluate pipeline directly, outside of any MCP tool call or

agent interaction — evidence that the pipeline was validated end-to-end as a plain console workflow before being decomposed into the eight independently callable tools in Table I. Second, the server's dependency list (`Image_Reconstruction_Classifier.csproj`) pulls in `ModelContextProtocol.AspNetCore` 2.2.0 for the MCP transport, `NeoCortexApi` 1.1.5 and `LearningApi` 1.3.1 for the HTM implementation, `Azure.Storage.Blobs` 12.29.1 for image persistence, `Azure.Data.Tables` 12.11.0 for the Table Storage-backed result persistence described in Section III-B, and `ClosedXML` 0.105.1 for the parallel Excel export described in Section III-B. The agent project depends on `Microsoft.Agents.AI` 1.17.0, `Microsoft.Extensions.AI(.OpenAI)` 10.9.0, and the matching `ModelContextProtocol` 2.2.0 client package, confirming that both sides of the MCP boundary in Fig. 1 are pinned to the same protocol version.

A separate `Tools_UnitTests` solution folder holds six MSTest projects (one per tested tool), together comprising six `[TestClass]` classes, 25 `[TestMethod]` tests, and 32 `Assert` calls, covering `ImageLoaderTool`, `ImageProcessorTool`, `ImageSpatialTool`, `HtmClassifierTool`, `KnnClassifierTool`, and `BinaryToImageConverterTool`; `ImageSimilarityTool` and `ImageFilterTool` do not yet have a corresponding test project.

F. Scalability Considerations

The agent-to-server relationship described in Section III-A is also the system's main scaling seam. Because `ImageReconstructionAgent` only depends on the `MCP_SERVER_URL` endpoint and never calls the reconstruction pipeline directly, any number of agent instances – or a human operator, or a test harness – can point at the same MCP server, and the server itself could in principle be deployed with multiple replicas behind Azure App Service's or Azure Container Apps' built-in autoscaling rules. Horizontal scaling was previously blocked by a purely in-memory classifier cache: each replica would have trained and held its own independent copy of the classifiers, so two requests routed to different replicas could have seen inconsistent state after a cold start. Persisting trained HTM/KNN model state as JSON in Blob Storage and reconstruction metrics in Table Storage (Section III-B) removes that specific blocker in principle – a new replica can read the same persisted model and result data that any other replica reads, rather than needing shared in-memory state – but this has not yet been exercised under an actual multi-replica deployment: concurrent-write behavior on `ReconstructionResults` under load, and the latency of loading a persisted model from Blob Storage on a cold start, remain unmeasured. Once that is validated, the tool-based decomposition in Table I also opens the door to scaling individual pipeline stages independently rather than the whole server uniformly – for example, allocating more replicas to the compute-heavy `ImageSpatialTool` (spatial pooling) than to the comparatively cheap `BinaryToImageConverterTool`, which a monolithic scaling strategy could not do.

TABLE III
AVERAGE RECONSTRUCTION SIMILARITY BY CLASSIFIER (N=2,000 TEST IMAGES). BINARY-SIMILARITY STANDARD DEVIATION WAS REPORTED ONLY FOR THE FUSION CLASSIFIER (7.05%) IN THE SOURCE EVALUATION.

Classifier	Vector Sim. (%)	Vector SD (%)	Binary Sim. (%)
KNN	83.39	10.06	87.00
HTM	85.90	7.95	87.28
Fusion (HTM+KNN)	86.22	8.06	88.12

IV. RESULTS

A. Reconstruction Accuracy

Table III summarizes similarity results across the full 2,000-image test set for each classifier. Vector similarity is the cosine similarity between the original and reconstructed pixel vectors; binary similarity is the fraction of matching pixel values after thresholding. These figures were produced by the offline `LocalPipelineRunner.cs` batch pipeline described in Section III-C, run once over the full training/test split; they are not a live measurement of the deployed MCP server, which does not yet expose the fusion step as a callable tool (Section III-A). The fusion classifier outperformed both individual classifiers on both metrics.

Counted directly from that pipeline's file-writing logic (`LocalPipelineRunner.cs`, `ImageSpatial.cs`): three preprocessing files (binarized, vectorized, spatial) per training image (30,000 across 10,000 images), plus the same three per test image plus a reconstructed vector and binary file for each of the HTM, KNN, and combined outputs (20,000 across 2,000 images) – a computed total of just over 50,000 artifacts, before the small attrition from images skipped on a logged parsing error. This offline evaluation corpus, distinct from and larger than the live Blob Storage object count in Section IV-C, is the artefact count this paper's Cloud Computing corpus-size figure refers to.

Three observations follow from Table III. First, the combined model achieved the highest accuracy on both metrics, confirming that fusing the two classifiers' outputs is preferable to using either alone. Second, HTM outperformed KNN in vector similarity (85.90% vs. 83.39%), consistent with its strength in capturing high-level, distributed feature representations rather than exact pixel matches. Third, KNN's lower vector similarity but competitive binary similarity (87.00%) suggests its direct pixel-based retrieval still contributes meaningfully to reconstruction quality even though it is the simpler of the two classifiers.

Fig. 5 and Fig. 6 characterize the distribution of similarity scores achieved by the fusion classifier across the full test set. The box plot shows a median vector similarity of approximately 90% (interquartile range 87%–94%) and a median binary similarity of approximately 88% (interquartile range 85%–91%), with a small number of low-scoring outliers. The

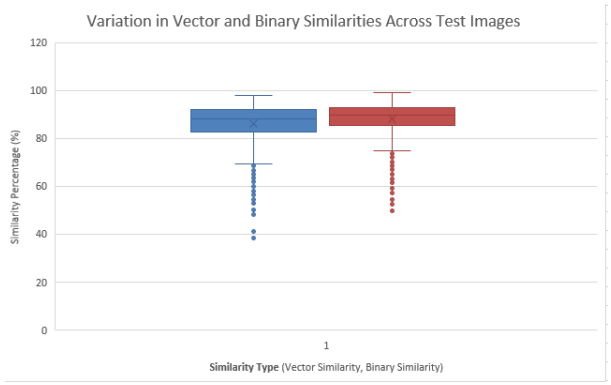

Fig. 5. Box plot of vector and binary similarity scores for the fusion classifier across the test set.

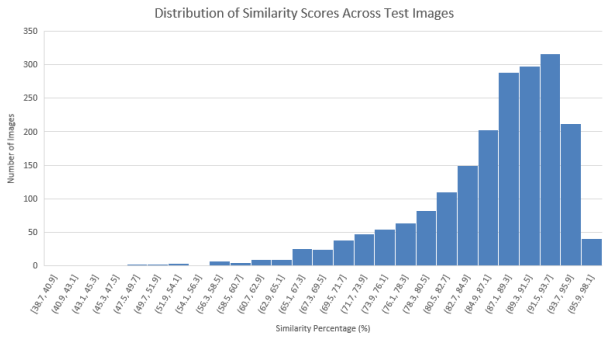

Fig. 6. Histogram of fusion-classifier similarity scores across the test set.

histogram confirms this: the majority of reconstructed images fall in the 80%–100% similarity range, with a pronounced peak between 85% and 90%, and only a thin tail of images below 60% similarity.

A case-study image (a handbag with subtle grayscale shading and low edge contrast) illustrates that tail: it produced markedly lower scores across all methods (KNN: 50.64% binary / 50.40% vector; HTM: 52.81% binary / 61.74% vector; Fusion: 49.87% binary / 58.07% vector), indicating that low-contrast, low-edge-density inputs remain a weak point of the current pipeline regardless of classifier choice. Fig. 7 shows a representative original-vs-reconstructed comparison for a higher-scoring test sample.

B. Computational Performance

Using `System.Diagnostics.Stopwatch`, the end-to-end pipeline — SDR generation via the Neocortex API, training both classifiers on 10,000 images, and testing on 2,000 images — completed in approximately 15 minutes on a standard local PC. This baseline predates the Azure-hosted deployment.

This local figure is the only latency measurement currently available. A re-measurement against the deployed Azure-hosted MCP server is still pending: it would need to isolate three cloud-specific costs that the local baseline does not incur — network round-trip time from the agent process to the hosted MCP endpoint, per-request Blob Storage read/write

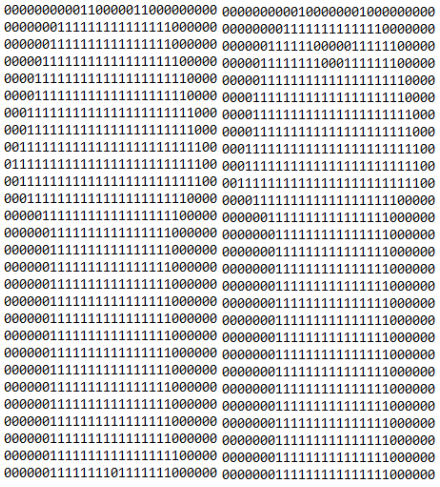

Fig. 7. Example original vs. HTM-reconstructed binary representation for a test sample.

latency for the `train`/`test` containers, and the one-time cost of repopulating the in-memory classifier cache (Section III-F) after a cold container start. None of these have been measured yet, so no cloud-hosted figure is reported here rather than an estimated one.

C. Cloud Deployment Evidence

The documentary evidence the guideline asks for beyond the architecture description itself has since been captured directly from the Azure Portal: a resource-group screenshot for `RG-ImageReconstruction` (Germany West Central) listing the `imgreconstructioncr` container registry and `imgreconstructionstr` storage account; the storage account's Overview and Access Keys blades confirming its name, region, and StorageV2/LRS configuration; a Containers listing confirming exactly `train` and `test` plus the platform-managed `$logs` container; the registry's Overview blade (login server `imgreconstructioncr.azurecr.io`, Basic tier); and the Web App's Overview blade (`image-reconstruction-application`, plan `AnnApiPlan`, Basic B1, Linux). That last screenshot also corrected two facts this section had carried from notes rather than the live system: the deployed image tag is `:v1`, not `:v2`, and the Portal reports the runtime status as "Issues Detected" rather than healthy — corrected here rather than left standing, per this paper's practice of disclosing a mismatch rather than erasing it (Section III-B).

Request-level evidence has since been captured as well. Connecting `ImageReconstructionAgent` to the live Azure-hosted endpoint (`image-reconstruction-application-gadkh2grhtfug8cp.germanywestcentral-01.azurewebsites.net`) succeeded: the agent reported loading 25 MCP-callable tools from the running server, and a natural-language request — "how many images we have in train container" — was answered correctly ("There are 10,000 images in the train container"), demonstrating that

the deployed service reads real Blob Storage data end-to-end despite the "Issues Detected" status above. A separate server console log confirms `ResultStorageService` connecting to the `ReconstructionResults` table on startup, though that particular log was captured from a local run against the same Table Storage account rather than from the App Service's own log stream. Still outstanding: a populated `ReconstructionResults` listing with actual similarity-score rows written by a live reconstruction call, and root-causing the "Issues Detected" status itself.

V. AI Usage & Discussion

The course guideline (Section 2.2.2) requires that every significant AI-assisted contribution be disclosed with its prompt, the tool and version used, a summary of the raw output, the manual change made, and the reason for that change. Rather than inline an abbreviated table with paraphrased or reconstructed prompt text, the author team is maintaining the complete, verbatim, per-prompt record separately in the project's `PROMPTS.md` file, which the guideline permits as the source of truth for this disclosure; paraphrasing prompts from memory into a paper table risks misrepresenting what was actually asked, which would itself run counter to the disclosure requirement. Table IV instead summarizes, at a categorical level, where AI involvement occurred in this project and which section or artifact discloses it, so a reader can navigate from this paper to the underlying evidence. The table's fourth row was cross-checked directly against `PROMPTS.md`'s complete 20-entry log rather than left as an open question: no entry documents an AI prompt for the HTM/KNN fusion weighting or similarity-scoring logic, so – consistent with `PROMPTS.md`'s role as the exhaustive record of every significant AI-assisted contribution – that logic is stated below as manually authored rather than hedged.

TABLE IV
Categories of AI Involvement in This Project

Category	Example Component(s)	Disclosed In
Run-time agent tool-calling	`Image-Reconstruction-Agent` (`Microsoft.-Agents.AI`, `gpt-4o-mini`) invoking MCP tools	Section III-A, III-D
AI-assisted coding during development	MCP tool scaffolding, classifier and pipeline helper code	`PROMPTS.md` (full log)
AI-assisted debugging / correction	Fixes to AI-suggested cloud SDK usage and naming/config issues identified in Section III-D	`PROMPTS.md`; Section III-D
Manually authored, algorithm-critical logic	HTM/KNN fusion weighting, similarity scoring (no `PROMPTS.md` entry documents AI assistance for this logic)	Section III-C

VI. Discussion

The reconstruction pipeline's main remaining limitation is algorithmic rather than architectural: it is sensitive to low-contrast, low-edge-density inputs, as the handbag case study in Section IV-A shows, with similarity dropping by roughly 30–35 percentage points relative to the dataset average, consistent with weaker SDR formation when binarization discards subtle grayscale detail. Architecturally, an earlier draft of this system had a genuine gap here – trained classifiers lived only in an in-memory cache and per-image metrics were only exported to Excel, so nothing survived a server restart – but that gap has since been closed by persisting classifier models to Blob Storage and reconstruction metrics to the `ReconstructionResults` table described in Section III-B; Section III-F discusses what is and is not yet validated about that fix. On the positive side, the MCP-based decomposition of the pipeline into independently callable tools worked well as an engineering pattern: it allowed the same reconstruction stages to be invoked either directly or through the AI agent without duplicating logic, and it kept each stage's responsibility (loading, processing, spatial pooling, classification, similarity, conversion) easy to reason about in isolation.

Two concrete examples, both visible directly in the codebase rather than reconstructed from memory, illustrate the kind of AI-output review this course's guideline asks for (Section 2.2.2, "depth of manual review/correction shown"). First, a manual review pass during development found that `ImageReconstructionAgent`'s system prompt instructed the agent to "...and save results to Azure Table Storage" while `ResultStorageService`/`ReconstructionResultEntity` was not yet being called from the reconstruction pipeline – the agent was, at that point, promising a persistence step the running system did not perform. This is exactly the kind of AI-authored-looking mismatch a manual pass is meant to catch: identifying it is what led to actually wiring `ResultStorageService` into the pipeline (Section III-B, Fig. 1), so the system prompt now describes real behavior rather than an aspirational one. Because the team's chat/commit history has not yet been cross-referenced to confirm exactly who or what introduced the original mismatch, that attribution is left open even though the fix itself is confirmed. Second, the configuration inconsistency noted in Section III-D — the `Training_Image_Spartial` folder name together with hardcoded Windows-style paths in `launchSettings.json` — is the kind of low-severity but real defect that AI-assisted scaffolding can introduce and that a manual review pass is expected to catch; it is documented rather than silently fixed, since deciding whether to rename a path that other configuration or scripts may depend on is a team decision outside the scope of this paper. A previously-noted file-naming inconsistency (`Tools/binaryToImageConverterTool.cs`'s lowercase-leading name) has since been corrected and

is no longer an open item.

More broadly, integrating an AI-scaffolded pipeline (NeoCortexApi spatial pooling, an ASP.NET-hosted MCP server, and Azure Blob Storage access) surfaced the same lesson twice: AI-generated code compiled and looked idiomatic, but keeping a persistence promise made in one file (the agent's system prompt) consistent with what another file actually implements (`ResultStorageService`) required a human reading both together — a cross-file consistency check that line-by-line AI-assisted code review does not automatically perform.

More generally, combining AI-assisted coding with cloud engineering surfaced a recurring pattern worth noting: AI suggestions for cloud SDK usage (connection strings, client construction, retry/error handling) tended to be syntactically plausible but required manual verification against current Azure SDK documentation before being trusted, since SDK versions and recommended patterns change frequently and AI suggestions can lag behind them — the `ResultStorageService` class sitting unwired for a period before being connected to the live pipeline is one concrete artifact of exactly this kind of AI-assisted scaffolding initially outrunning what was actually integrated and tested.

Threats to Validity: The accuracy results in Section IV-A were measured entirely on Fashion-MNIST, a controlled 28×28-pixel grayscale dataset; they should not be assumed to generalize to higher-resolution or color imagery without further evaluation. The 15-minute runtime baseline (Section IV-B) was measured on a single local machine and has not yet been reproduced on the target Azure hosting environment, where network latency to Blob Storage and container cold-start behavior may materially change throughput. The cost of loading a persisted classifier model from Blob Storage on a cold start (Section III-F) has also not been measured, so it is not reflected in the local baseline either. Finally, the AI-agent orchestration layer (`ImageReconstructionAgent`) has only been exercised against a single model family (OpenAI `gpt-4o-mini`); its behavior with other models is untested.

VII. Conclusion and Future Work

This paper presented a hybrid HTM/KNN image reconstruction pipeline re-engineered as a containerized service, deployed to Azure App Service and exposed through Model Context Protocol tools operable by an AI agent, achieving an average vector similarity of 86.22% and binary similarity of 88.12% on a 2,000-image Fashion-MNIST test set with the fusion classifier outperforming either individual classifier. Beyond the algorithmic result, the paper documents — and, where evidence is still outstanding, explicitly flags — the cloud architecture and AI-assisted engineering process behind the system, including a candid account of a persistence gap (in-memory-only classifier state and metrics) that was identified during manual review and subsequently closed by wiring up Blob-Storage-backed model persistence and Table-Storage-backed metrics, in line with this course's emphasis on treating both architecture and AI usage as disclosed and reviewed engineering practice rather than hidden implementation detail.

Future work includes exposing the confidence-weighted HTM/KNN fusion step itself as a live MCP tool call or a documented agent-orchestrated sequence, so the accuracy figures reported here can eventually be reproduced against the deployed server rather than only the offline batch pipeline; improving reconstruction quality for low-contrast, low-edge-density images through adaptive binarization or contrast enhancement; expanding the training and test sets to more diverse garment types and image conditions; and, on the engineering side, validating the newly-closed persistence path under an actual multi-replica deployment: concurrent writes to `ReconstructionResults`, cold-start latency for loading a persisted model from Blob Storage, and replica-to-replica consistency have not yet been measured, and are the prerequisite for confidently claiming the scaling path laid out in Section III-F. Remaining future work includes adding automated CI/CD deployment to Azure, monitoring and alerting for the hosted MCP server, capturing the populated `ReconstructionResults` listing still outstanding per Section IV-C and diagnosing the "Issues Detected" runtime status reported there, and cost/performance profiling of the Blob and Table Storage access patterns under sustained, multi-replica, agent-driven load.

References

[1] J. Pokorný, "Big data storage and management: Challenges and opportunities," in *Environmental Software Systems. Computer Science for Environmental Protection*, 12th IFIP WG 5.11 Int. Symp., ISESS 2017, Proc., Zadar, Croatia, 2017, pp. 28–38, Springer.

[2] X. Chen, W. Wang, and W. Li, "An overview of Hierarchical Temporal Memory: A new neocortex algorithm," in *Proc. Int. Conf. Modelling, Identification and Control (ICMIC)*, Wuhan, China, 2012, pp. 1004–1010, IEEE.

[3] Y. Cui, S. Ahmad, and J. Hawkins, "The HTM spatial pooler—A neocortical algorithm for online sparse distributed coding," *Frontiers in Computational Neuroscience*, vol. 11, p. 111, 2017.

[4] Y. Cui, C. Surpur, S. Ahmad, and J. Hawkins, "A comparative study of HTM and other neural network models for online sequence learning with streaming data," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, Vancouver, Canada, 2016, pp. 1530–1538, IEEE.

[5] H. A. Abu Alfeilat, A. B. A. Hassanat, O. Lasassmeh, A. S. Tarawneh, M. B. Alhasanat, H. S. Eyal Salman, and V. B. S. Prasath, "Effects of distance measure choice on k-nearest neighbor classifier performance: A review," *Big Data*, vol. 7, no. 4, pp. 221–248, 2019.

[6] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," *arXiv:1708.07747*, 2017.

[7] D. Dobric, "NeoCortexApi: C#.NET implementation of the Hierarchical Temporal Memory cortical learning algorithm," v1.1.5, GitHub repository, 2024. [Online]. Available: https://github.com/ddobric/neocortexapi

[8] Model Context Protocol, "Specification," 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-11-25

[9] Microsoft, "Introduction to Blob (object) storage - Azure Storage," Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction

[10] Microsoft, "Introduction to Table storage - Azure Storage," Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/storage/tables/table-storage-overview

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," *arXiv:2210.03629*, 2023.

[12] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *arXiv:2302.04761*, 2023.